BINV2150-A — Javascript : avancé

Séance 19 - JSON Web Tokens

Haute École Léonard de Vinci

Authentification backend

- Tout le monde peut envoyer des requêtes au backend, pas seulement le frontend !
- Certaines requêtes doivent être restreintes à certains utilisateurs
- Comment authentifier l'utilisateur qui effectue une requête ?

JSON Web Tokens (JWT)

- Jeton au format JSON utilisé pour authentifier et sécuriser les communications entre frontend et backend
- 3 parties : `header.payload.signature`
  - **Header** : Type et algorithme de signature
  - **Payload** : Données (userId, rôle, expiration...)
  - **Signature** : Garantit l'authenticité et l'intégrité
- Header et Payload encodés au format Base64, **pas chiffré** ➡️ pas de données sensibles !
- Signature chiffré avec une clé de chiffrement
- Workflow :
  - Le frontend contacte le backend à une route `/login`
  - Le backend génère un JWT sécurisé et le renvoie au frontend
  - Le frontend stocke de manière sécurisée ce JWT
  - Pour chaque requête devant être identifiée, le frontend envoie ce JWT dans le header d'authorisation
  - Le backend reçoit le JWT avec chaque requête et vérifie la signature pour autoriser l'accès
- Un JWT est une information sensible ! Si quelqu'un récupère un de vos token, il peut se faire passer pour vous !

Structure d'un JWT

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJ1c2VybmFtZSI6ImFkbWluIiwiaWF0IjoxNzYzNDgwNTU0LCJleHAiOjE3NjM1NjY5NTR9.
ybbQOGJJOzoVFhBZW0UiIx6Zq-wIqBmCv7hsxlPziwI
```

https://www.jwt.io

Librairie `jsonwebtoken`

- Installation

```
npm i jsonwebtoken
npm i -D @types/jsonwebtoken
```

- Configuration

```
// import de la librairie
import jwt from "jsonwebtoken";

// Définition du secret utilisé pour chiffrer la signature
// Valeur à garder secrète, ne pas partager sur git !
const jwtSecret = "";

// Définition du temps de validité du token en secondes
// validité d'un jour : 24 (heures) * 60 (minutes) * 60 (secondes)
const jwtLifetime = 24 * 60 * 60;
```

Génération et vérification de JWT

- Génération

```
const token = jwt.sign(
    { username }, // objet à inclure dans le payload du JWT
    jwtSecret, // clé utilisée pour générer la signature du JWT
    { expiresIn: jwtLifetime }, // options de génération du JWT
);
```

- Vérification

```
try {
    const { username } = jwt.verify(token, jwtSecret) as JwtPayload;
    // JWT correctement vérifié
} catch (err) {
    // Impossible de décoder le JWT
    // ou signature incorrecte
    // ou JWT expiré
    // ...
}
```

Types

```ts
// types.ts
interface Credentials {
  username: string;
  password: string;
}

interface User {
  id: number;
  username: string;
  password: string;
}

interface JwtPayload extends jwt.JwtPayload {
  username: string;
}

interface AuthenticatedRequest extends Request {
  user?: User;
}
```

Services

```ts
// services/auths.ts
function getUser(username: string): User | undefined {
  return parse<User>(jsonDbPath, defaultUsers).find((user) => user.username === username);
}

function register(credentials: Credentials): boolean {
  const userFound = getUser(credentials.username);
  if (userFound) return false; // User already exists

  const users = parse<User>(jsonDbPath, defaultUsers);
  const newId = users.length > 0 ? users[users.length - 1].id + 1 : 1;
  const newUser: User = {
    id: newId,
    username: credentials.username,
    password: credentials.password,
  };
  users.push(newUser);
  serialize(jsonDbPath, users);

  return true;
}
```

Services

```ts
// services/auths.ts
const jwtSecret = "secret";
const lifetimeJwt = 24 * 60 * 60; // 24h

function login(credentials: Credentials): string | undefined {
  const user = getUser(credentials.username);
  if (!user) return undefined; // User not found
  if (user.password !== credentials.password) return undefined; // Invalid password

  const token = jwt.sign({ username: user.username }, jwtSecret, { expiresIn: lifetimeJwt });
  return token;
}

function verify(token: string): string | undefined {
  try {
    const { username } = jwt.verify(token, jwtSecret) as JwtPayload;
    return username; // Token is valid, return username
  } catch (error) {
    return undefined; // Token is invalid or expired
  }
}
```

Routes

```ts
// routes/auths.ts
router.post("/register", (req, res) => {
  const credentials: unknown = req.body;
  if (!isCredentials(credentials)) return res.sendStatus(400); // Invalid input

  const created = register(credentials);
  if (!created) return res.sendStatus(409); // User already exists

  return res.sendStatus(201);
});

router.post("/login", (req, res) => {
  const credentials: unknown = req.body;
  if (!isCredentials(credentials)) return res.sendStatus(400); // Invalid input

  const token = login(credentials);
  if (!token) return res.sendStatus(401); // Invalid username or password

  return res.json({ token });
});
```

Middlewares

```ts
// utils/auths.ts
const authorize = (req: AuthenticatedRequest, res: Response, next: NextFunction) => {
  const token = req.get("authorization");
  if (!token) return res.sendStatus(401); // No token provided

  const username = verify(token);
  if (!username) return res.sendStatus(401); // Invalid token

  const user = getUser(username);
  if (!user) return res.sendStatus(401); // User not found

  req.user = user;
  return next();
}

const isAdmin = (req: AuthenticatedRequest, res: Response, next: NextFunction) => {
  if (req.user!.username !== "admin") return res.sendStatus(403); // Forbidden
  return next();
};
```

App

```ts
// app.ts
app.use("/auths", authsRouter);

app.get("/", (_req, res) => {
  res.send(`Welcome to the JWT Demo App`);
});

app.get("/profile", authorize, (req: AuthenticatedRequest, res) => {
  const user = req.user!;
  res.send(`Hello, ${user.username}! This is your profile page.`);
});

app.get("/admin", authorize, isAdmin, (_req, res) => {
  res.send("Welcome, Admin! This is a protected admin route.");
});
```

Tests

```
# REST Client/tests.http

### Login the default admin
# @name admin
POST {{baseUrl}}/auths/login
Content-Type: application/json

{
    "username":"admin",
    "password":"admin"
}

### Login the manager user
# @name manager
POST {{baseUrl}}/auths/login
Content-Type: application/json

{
    "username":"manager",
    "password":"manager"
}
```

Tests

```
# REST Client/tests.http

### unprotected route
GET {{baseUrl}}

### protected route (admin or manager)
GET {{baseUrl}}/profile
Authorization: {{manager.response.body.token}}

### protected route (admin only)
GET {{baseUrl}}/admin
Authorization: {{admin.response.body.token}}
```